COE 379L Midterm Project Report: Atlas Laboratory Agent

Name: *Ayush Puthiyavettle* | UT EID: *ap63642* | Repository: *https://github.com/Pvayush/coe379l-midterm1*

1. System Overview

Atlas treats model output as a proposal. The trusted LabRequest fixes the authenticated user and resolved project, resource, and interval. The supplied initial message format labels that envelope and the original request text separately. The system prompt treats retrieved passages and observations as untrusted data. It considers the data as evidence, but embedded commands cannot override the request or execution rules.

At each step, Atlas asks the model for one decision, and Pydantic uses the kind field to check that the response has the right structure. If the model needs clarification, Atlas returns its question without calling a tool. If it proposes a tool call, Atlas first validates the arguments and checks whether the action is allowed. These checks ensure the arguments match the trusted request, require all ten facts supplied by ReservationContext for reservations, and prevent a second successful reservation in the same run.

Approved calls go to the supplied dispatcher. Every proposed tool call, including one that is denied, gets a TraceEvent recording the decision, observation, step number, and whether the tool can change state. Atlas then adds the serialized decision and observation to the conversation so the model can use the result when choosing its next step.

Before returning a final response, Atlas verifies that its citations come from retrieved sources, any claimed changes succeeded, and its status matches the reported completed actions. By default, a run stops after ten decisions. The supplied simulator, tools, retrieval, context computation, and evaluation modules remain unchanged.

2. Public Benchmark Results

Table 1. Aggregate metrics from public_benchmark.json.

Metric	Value	Metric	Value
`scenario_count`	6	`unnecessary_refusal_rate`	0
`success_rate`	1	`clarification_accuracy`	1
`invalid_structured_response_rate`	0	`average_tool_calls`	2.66
`unsafe_action_attempt_rate`	0.0		

The tensile-tester case succeeds after checking policy, user authorization, and equipment status, then making one reservation. Four other cases are blocked after three read-only tool calls each, which are the missing laser training, expired robot calibration, missing after-hours approval, and an overlapping booking. An ambiguous request makes the agent ask for clarification without calling any tools. Across all cases, there are 16 trace events, and every public test produces the expected outcome.

The public unsafe-attempt metric counts runs where the action gate rejects a proposed call; it does not count unsafe actions that actually execute. These scripts never attempt an unsafe change, so a zero rate does not, by itself, show that the gate works. For example, the booking-conflict script stops when it learns the equipment is unavailable and never tries to reserve it. All results come from predefined ScriptedLLM responses, rather than live model inference.

3. Student Tests

The full pytest suite passes 26 cases: 23 supplied and three added cases in tests/test_student_cases.py. All three added tests use ScriptedLLM and assert specific response, trace, or execution properties.

The first test proposes a reservation before any checks. It runs just one decision and checks that the trace records an action-gate denial and that the simulator still contains only its initial reservation. The proposed interval has no conflict, so an executed reservation would add a booking. These assertions check that the unsafe proposal produces a denial without changing reservation state.

The second test immediately supplies a final response claiming that a reservation was completed, although no tool has run. It checks that the agent returns a failed response with an invalid-final-response termination reason. The third test supplies an unknown decision kind and checks for an invalid-model-decision termination reason and an empty trace.

4. Implementation Decisions

The validators report unmet requirements, block reservations with missing trusted details, and match resolved values exactly. The prompt instructs the model to ask for clarification when details are missing. The LLM adapter validates each decision, while tool schemas check arguments before they reach the context helper. Invalid dates and times produce recorded failure observations.

Termination reasons distinguish completion, clarification, invalid decisions, inconsistent final responses, and the decision limit. Failed or denied tool calls inform the next model decision. A later controller failure does not undo a successful reservation: the result remains in the trace, although the failure response does not explicitly report that success. Further reservations are blocked.

These checks provide limited guarantees. Retrieved citations may not support the answer, and validation checks structured action claims rather than every sentence. The supplied context helper’s hour-based approval requirement treats end times of 18:00 and 18:30 alike. Observations can become stale, policy checks and reservations are not atomic, and there is no automatic rollback. Scripted tests do not establish exhaustive coverage, complete injection resistance, or live-model reliability.

Data Flow Diagram (not counted toward the 2-page limit)

